

DevOps Interview Q&A Guide

Junior DevOps Engineer · Cloud Engineer · SRE · Platform Engineer

38 Real Interview Questions with Structured Answers

Sanket Ajay Chopade | B.Tech Aeronautical Engineering → DevOps

1. EKS & Kubernetes

Q: How would you handle an EKS production issue?

Immediate triage flow:

- Check pod status: `kubectl get pods -n` — identify CrashLoopBackOff, OOMKilled, Pending
- Describe the failing resource: `kubectl describe pod` for events and error messages
- Pull logs: `kubectl logs --previous` for crash logs
- Check node health: `kubectl get nodes` — look for NotReady, pressure conditions
- Verify HPA/resource limits aren't throttling the pod
- Check CloudWatch Container Insights and ALB target group health
- If node-level: SSH into node via SSM, check kubelet status, disk/memory pressure
- Escalate: snapshot state, page on-call if SLO breach imminent, open postmortem

■ *Always preserve evidence before restarting pods in production.*

Q: What would you do if Kubernetes nodes show NotReady status?

Diagnosis steps:

- `kubectl describe node` — check Conditions section for MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable
- SSH into the node (via SSM/bastion) and check: `sudo systemctl status kubelet`
- View kubelet logs: `sudo journalctl -u kubelet -n 100`
- Verify node can reach the API server and has valid credentials
- Check `/var/log/messages` or `dmesg` for kernel OOM events
- If AWS: verify ENI limits, check EC2 health in console, check CNI plugin (aws-node DaemonSet)

Q: What are the most common reasons for NotReady nodes?

- kubelet crashed or stopped (check systemd service)
- Node ran out of memory or disk — kubelet self-evicts
- Network plugin (CNI) failure — aws-node pod crashing on EKS
- Node lost connectivity to API server (security group / NACLs)
- Certificate expiry on older self-managed clusters
- EC2 hardware failure or AWS underlying host issue

Q: How would you deploy changes to 200 pods with zero downtime?

- Use RollingUpdate strategy in the Deployment spec (default in K8s)
- Set `maxUnavailable: 0` and `maxSurge: 25%` — ensures old pods stay up while new ones start

- Use readinessProbe correctly — K8s won't route traffic until probe passes
 - Use PodDisruptionBudget (PDB) to enforce min available during node drains
 - Consider canary rollout via Argo Rollouts or Flagger for risk mitigation
 - Monitor rollout: `kubectl rollout status deployment/`
 - Rollback instantly if needed: `kubectl rollout undo deployment/`
-

Q: How do you determine Kubernetes resource requests and limits?

- Start with VPA (Vertical Pod Autoscaler) in recommendation mode — it observes actual usage
 - Check Prometheus metrics: `container_cpu_usage_seconds_total`, `container_memory_working_set_bytes`
 - Set requests = p50 of observed usage (what pod normally needs)
 - Set limits = p95-p99 (burst tolerance without OOM risk)
 - Never set limits on CPU unless necessary — CPU throttling is silent; OOMKill is loud
 - Re-evaluate after load testing with realistic traffic patterns
-

Q: How do you troubleshoot OOMKilled pods?

- `kubectl describe pod` — check Last State: OOMKilled in container status
 - Check memory limit vs actual usage in Prometheus/Grafana
 - `kubectl top pod` for live usage
 - If limit is too low → increase it (with evidence from metrics)
 - If app has a memory leak → fix the application, not just the limit
 - Check JVM heap settings, Python memory management, connection pool leaks
-

Q: Would you choose one 8GB pod or multiple smaller pods and why?

Multiple smaller pods is almost always correct. Reasons:

- Resilience: one large pod = single point of failure; many pods = tolerate individual failures
 - Scheduling: large pods are harder to place, risk node fragmentation
 - Scaling: horizontal scaling (HPA) works on smaller pods; you can't split an 8GB pod
 - Blast radius: crash in one small pod doesn't affect others
 - Exception: stateful workloads (DBs, in-memory caches) may legitimately need one large pod
-

Q: What if Apache inside a Kubernetes pod is consuming excessive CPU?

- `kubectl exec -it -- top` to confirm Apache process is the culprit
- Check Apache access logs for traffic spike or runaway requests
- Look for infinite loops, slow SQL queries being hit repeatedly
- Check if HPA is configured — if not, manually scale replicas: `kubectl scale deploy --replicas=N`
- Set CPU limits if absent; use LimitRange at namespace level
- If persistent: profile the application, check `MaxRequestWorkers` in Apache config

2. AWS & Cloud Infrastructure

Q: Have you used AWS Lambda for infrastructure automation? Give an example.

Example: Automated nightly EC2 stop/start at Hisan Labs:

- Lambda function triggered by EventBridge (cron schedule) at 11 PM
- Function reads EC2 instances by tag: Environment=dev, Action=auto-stop
- Calls `ec2.stop_instances()` for tagged instances; skips production-tagged ones
- Sends SNS notification with list of stopped instances for audit trail
- Another Lambda at 7 AM starts them back up
- IAM role with minimal permissions: `ec2:StopInstances`, `ec2:StartInstances` scoped to resource tags

Q: How would you stop 46 out of 50 EC2 instances every day while keeping 4 running?

- Tag the 4 permanent instances: `KeepRunning=true`
- Lambda function: describe all instances → filter out `KeepRunning=true` → stop the rest
- Trigger via EventBridge rule: `cron(0 18 * * ? *)` for 6 PM stop
- Add error handling: log skipped instances, alert on API errors via CloudWatch Alarms

```
import boto3
ec2 = boto3.client('ec2')
response = ec2.describe_instances(
    Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]
)
to_stop = []
for r in response['Reservations']:
    for i in r['Instances']:
        tags = {t['Key']: t['Value'] for t in i.get('Tags', [])}
        if tags.get('KeepRunning') != 'true':
            to_stop.append(i['InstanceId'])
if to_stop:
    ec2.stop_instances(InstanceIds=to_stop)
```

Q: Would you use Instance IDs or Tags for automation and why?

Tags, always. Reasons:

- Instance IDs change when instances are replaced or new ones are created
- Tags are declarative — you express intent (`Environment=dev`) not identity
- Tags scale: add a new instance with the right tag and automation picks it up automatically
- Tags enable cost allocation, access control (IAM tag conditions), and filtering
- Instance IDs are only appropriate for one-off, targeted operations

Q: How would you enable cross-account access between Lambda and S3?

- S3 bucket policy (in Account B) grants access to Lambda's role ARN from Account A
- Lambda's execution role (Account A) must have `s3:GetObject` / `s3:PutObject` permissions
- For KMS-encrypted buckets: also grant `kms:Decrypt` to Lambda's role in the key policy
- Test with `aws s3 ls s3://bucket --profile cross-account` before deploying Lambda

```
# S3 Bucket Policy (Account B)
{
  "Principal": {"AWS": "arn:aws:iam::ACCOUNT_A:role/LambdaRole"},
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3::my-bucket/*"
}
```

Q: What are VPC Endpoints?

VPC Endpoints allow private connectivity from your VPC to AWS services without traversing the public internet. Two types:

- Gateway Endpoint: for S3 and DynamoDB only. Free. Added as a route table entry. No ENI created.
- Interface Endpoint (PrivateLink): for most other AWS services (SSM, ECR, STS, etc.). Creates an ENI with a private IP in your subnet. Charged per hour + per GB.

Q: Difference between Gateway Endpoint and Interface Endpoint?

Key differences:

- Gateway: route-table based, free, only S3/DynamoDB, no DNS changes needed
- Interface: ENI-based, private IP in subnet, supports DNS resolution, works across VPCs via PrivateLink, costs money
- Use Gateway for S3/DynamoDB — always. Use Interface for everything else when you need private access.

Q: When would you use Transit Gateway instead of VPC Peering?

- VPC Peering: point-to-point, no transitive routing, fine for 2-4 VPCs
- Transit Gateway: hub-and-spoke model, transitive routing supported, scales to hundreds of VPCs
- Use TGW when: 5+ VPCs need to communicate, you need centralized routing control, or on-premises connectivity via one attachment
- TGW costs more but eliminates the $N*(N-1)/2$ peering complexity at scale

Q: Have you migrated workloads between cloud providers?

At Hisan Labs, we evaluated GCP migration for a batch processing workload. Steps followed:

- Inventory all services: EC2 → GCE, S3 → GCS, RDS → Cloud SQL
- Containerize stateless workloads first — portable across clouds
- Use Terraform with provider abstraction to minimize rewrite
- Data migration: AWS DMS or custom scripts with validation checksums
- Run parallel (shadow mode) for 2 weeks before cutover
- DNS cutover last — fastest rollback option if issues arise

Q: How do you automate AMI copy across AWS regions?

- Use `aws ec2 copy-image` CLI or `boto3 ec2.copy_image()`
- Lambda triggered by EventBridge or SNS from source region on new AMI creation
- Copy to target regions, wait for available state, then share or tag appropriately
- Use SSM Parameter Store to track latest AMI ID per region
- Enable AMI lifecycle policies in EC2 Image Builder for automated creation + cross-region copy

```
ec2_target = boto3.client('ec2', region_name='eu-west-1')
ec2_target.copy_image(
    Name='my-ami-copy',
    SourceImageId='ami-0abc123',
    SourceRegion='us-east-1'
)
```

3. Terraform & Infrastructure as Code

Q: How do you structure Terraform code for Dev, UAT and Production?

Recommended: separate state per environment with shared modules.

- Directory structure: environments/dev/, environments/uat/, environments/prod/ — each with main.tf, variables.tf, terraform.tfvars
- Shared logic lives in modules/: modules/vpc, modules/eks, modules/rds
- Each environment has its own remote state (S3 backend with DynamoDB lock), different variable values
- Use Terragrunt to DRY up backend config and enforce structure across environments
- Never promote code by copy-paste — parameterize everything in modules

Q: How do you handle Terraform state drift?

- Run terraform plan regularly (in CI) to detect drift — any diff between state and reality is drift
- terraform refresh updates state to match real-world (use carefully — it can cause unintended destroys)
- For drift caused by manual changes: import the resource (terraform import) or let Terraform reconcile on next apply
- Best prevention: restrict console access, enforce all changes via IaC (SCPs + approval gates)
- Alert on drift using Atlantis, Spacelift, or custom Lambda polling plan output

Q: What happens if someone manually deletes a Terraform-managed resource?

Terraform's state still references the resource. On next terraform plan:

- Terraform detects the resource is missing from the provider
- Plan shows it will recreate the resource
- terraform apply will recreate it — state becomes consistent again
- Risk: if the resource had dependencies (DNS records, IAM bindings), those may break temporarily
- Prevention: use Resource-Based Policies and SCPs to block console deletions of critical resources

Q: How do you rename Terraform resources without recreation?

Two options:

- terraform state mv old_resource_name new_resource_name — renames in state file without touching infrastructure
- moved {} block (Terraform 1.1+): declare the rename in config, Terraform handles it on next apply cleanly
- Never just rename in .tf file without one of these — Terraform will destroy old and create new

```
# Terraform 1.1+ moved block
moved {
  from = aws_instance.web_old
  to   = aws_instance.web_new
}
```

4. Docker & Containers

Q: How do you manage secrets inside Docker containers?

- Never bake secrets into images (ENV vars at build time are visible in image history)
- Pass at runtime via `--env-file` or environment variables injected by orchestrator
- Preferred: AWS Secrets Manager + ECS task definition secrets OR Kubernetes Secrets (base64, not plaintext)
- Better: External Secrets Operator (ESO) to sync AWS Secrets Manager → K8s Secrets automatically
- Docker Swarm: use Docker Secrets (mounted as tmpfs in container, not env vars)
- Audit image with `docker history --no-trunc` to verify no secrets leaked during build

Q: How do you reduce Docker image size?

- Use multi-stage builds: build in a full image, copy only the binary/artifact to a minimal base
- Use slim or alpine base images: `python:3.11-slim` vs `python:3.11` (saves 400MB+)
- Combine RUN commands to reduce layer count and clean up in the same layer
- Use `.dockerignore` to exclude `node_modules`, `.git`, test files
- Remove build tools after use: `apt-get purge -y gcc build-essential && apt-get autoremove`
- Use `docker scout` or `dive` to audit layer sizes

```
# Multi-stage example
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o app

FROM gcr.io/distroless/static
COPY --from=builder /app/app /app
ENTRYPOINT ["/app"]
```

Q: How do you troubleshoot ECS memory issues?

- CloudWatch Container Insights: `MemoryUtilized` vs `MemoryReserved` per task
- If `MemoryUtilized` consistently > 80% of `MemoryReserved` → increase task memory
- OOM events: check CloudWatch Logs for the task — ECS stops container and logs exit code 137
- For Fargate: memory is a hard limit — no cgroup headroom. Increase task definition memory.
- For EC2 launch type: check node-level memory with CloudWatch agent metrics
- Profile the application to find leaks before simply increasing the limit

5. Linux & System Troubleshooting

Q: If a Linux cron job is not running, how would you troubleshoot it?

- Check cron service: `sudo systemctl status cron` (or `crond` on RHEL/CentOS)
 - Verify crontab syntax: `crontab -l` — look for syntax errors, missing newline at end
 - Check system logs: `grep CRON /var/log/syslog` or `/var/log/cron`
 - Check if script is executable: `ls -la /path/to/script.sh` — needs `+x` bit
 - Verify the script runs manually as the same user cron would use
 - Check environment: cron has minimal PATH — use absolute paths in scripts
 - Check for output: add `>> /tmp/cron.log 2>&1` to capture stdout/stderr
 - Verify timing: use `crontab.guru` to validate the expression
-

Q: How do you identify which process is listening on port 8080?

- `sudo ss -tlnp | grep 8080` — preferred (faster than `netstat`)
 - `sudo netstat -tlnp | grep 8080` — classic alternative
 - `sudo lsof -i :8080` — shows process name and PID
 - Once you have PID: `ps aux | grep` for full command line
 - On container: `kubectl exec` or `docker exec` then run the same commands inside
-

Q: How do you troubleshoot a 'No Space Left on Device' error when disk space appears available?

This is the classic inode exhaustion problem:

- `df -h` shows disk % — but `df -i` shows inode usage
 - If inode usage is 100%, you cannot create new files even with free space
 - Find directories with many small files: `find / -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head`
 - Common culprit: log files, temp files, mail queues, Docker layer files
 - Clean up: `rm` small unnecessary files, rotate logs, docker system prune
 - Long term: tune filesystem inode ratio at creation (`mkfs.ext4 -i`) or use XFS (dynamic inodes)
-

Q: How would you troubleshoot a process consuming 100% CPU?

- `top` or `htop` — identify PID and process name
- `ps aux --sort=-%cpu | head` — confirm top consumers
- `strace -p` — see what syscalls the process is making (I/O loops, etc.)
- For Java: `jstack` to get thread dump, identify busy thread
- For Python: `py-spy top --pid` for live profiling
- Check if it's a known runaway (cron job, log rotation, backup process)
- If application bug: capture state, then `kill -9` as last resort

6. Incident Management & Monitoring

Q: Describe the worst production incident you owned and how you resolved it.

Example answer (structured with STAR):

- Situation: At Hisan Labs, a misconfigured Kubernetes HPA caused cascading pod restarts during peak hours — 60% of services became unavailable
- Task: I was the on-call engineer and owned the incident bridge
- Action: Identified HPA was scaling down due to incorrect CPU metric target (1% vs 70%). Immediately patched the HPA target, manually scaled up critical deployments, and rolled back the config change that introduced the error. Communicated status every 10 minutes to stakeholders.
- Result: Full restoration in 23 minutes. Conducted postmortem, added HPA change review to our deployment checklist, and set up alerts on replica count drops > 50%

Q: Which metrics do you monitor during incidents?

- Infrastructure: CPU utilization, memory usage, disk I/O, network throughput
- Kubernetes: pod ready count, restart count, HPA replica count, node conditions
- Application: request rate (RPS), error rate (5xx %), latency (p50/p95/p99)
- Database: query latency, connection pool saturation, replication lag
- Business: checkout success rate, API response times for critical endpoints
- Use the RED method (Rate, Errors, Duration) for services and USE (Utilization, Saturation, Errors) for resources

Q: What CPU thresholds do you use for scaling?

- HPA: typically trigger scale-out at 70% CPU — leaves headroom before saturation
- Scale-in: 30-40% to avoid thrashing (constant scale up/down)
- Node group ASG: scale at 80% — account for DaemonSet overhead
- These are starting points — tune based on your app's load profile and latency SLOs
- Use custom metrics (request queue depth, RPS) in addition to CPU for more accurate scaling signals

Q: What are your day-to-day responsibilities and how do you receive work requests?

At Hisan Labs:

- Work intake: Jira tickets from dev teams, Slack alerts from monitoring, and sprint planning for project work
- Daily: review CloudWatch dashboards, address any overnight alerts, stand-up with dev teams
- Project work: CI/CD pipeline improvements, Terraform module updates, K8s cluster upgrades
- On-call: 1-week rotation, respond to PagerDuty alerts, own incident resolution
- Documentation: update runbooks after every incident, maintain internal wiki

Q: What manual task have you automated that saved the most time?

EC2 environment cleanup (described earlier). Before automation:

- Dev team manually stopped ~46 EC2 instances every evening — 15-20 minutes of manual work, often forgotten
- Cost impact: instances running overnight unnecessarily — ~\$400/month wasted
- After Lambda automation: zero manual effort, consistent execution, SNS report for audit
- Saved estimated 5+ hours/week of developer time and eliminated the billing waste

7. Disaster Recovery & Resilience

Q: Have you performed Disaster Recovery drills?

- Yes — quarterly DR drills at Hisan Labs simulating full region unavailability
 - Drill scope: failover EKS workloads to secondary region, restore RDS from cross-region replica, validate DNS failover via Route 53 health checks
 - Each drill was time-boxed and documented — actual RTO/RPO measured against targets
 - Found gaps: one drill revealed Route 53 health check misconfiguration that would have delayed failover by 8 minutes — fixed immediately
-

Q: What were your RPO and RTO targets?

- RPO (Recovery Point Objective): 1 hour — maximum data loss tolerable. Achieved via hourly RDS snapshots + continuous binlog replication to read replica
 - RTO (Recovery Time Objective): 30 minutes — time to restore service. Achieved via pre-warmed standby environment + Route 53 failover routing
 - For critical services (payment): RPO 5 minutes, RTO 10 minutes — required synchronous replication
-

Q: How do you achieve near-zero data loss for production databases?

- RDS Multi-AZ: synchronous replication — failover with near-zero data loss, automatic
 - Aurora Global Database: < 1 second replication lag to secondary regions
 - Enable automated backups with 35-day retention + manual snapshot before major changes
 - Point-in-time recovery (PITR) for granular restore capability
 - For self-managed: binlog shipping with sub-second delay to replica
 - Test restores quarterly — untested backups are not backups
-

Q: How would you implement cross-region failover?

- DNS layer: Route 53 with health checks on primary endpoint; failover routing policy to secondary
- Data layer: RDS cross-region read replica (promote on failover) or Aurora Global DB
- Compute layer: pre-deployed EKS/ECS cluster in secondary region, possibly scaled to 0
- State/config: S3 with cross-region replication, Secrets Manager multi-region replication
- Trigger: automated (health check fails) or manual (runbook-driven during incidents)
- Document and test the runbook — failover under stress is not the time to figure it out

8. Load Testing & Performance

Q: How would you load test an application for 150 concurrent users?

- Tool: k6, Locust, or JMeter — k6 preferred for scripting flexibility and CI integration
- Define realistic user flows: login → browse → checkout (not just homepage hits)
- Ramp up: 0 → 50 → 100 → 150 users over 10 minutes, then sustain for 20 minutes
- Metrics to capture: p95 latency < 500ms, error rate < 1%, throughput RPS
- Monitor backend during test: CPU, memory, DB connections, pod scaling events
- Run from outside the VPC (simulate real users) and from within (measure app-only latency)
- 150 users is a relatively light load — also test 300-500 users to find your breaking point

```
// k6 example
import http from 'k6/http';
export let options = {
  stages: [
    { duration: '5m', target: 150 },
    { duration: '10m', target: 150 },
    { duration: '2m', target: 0 },
  ],
};
export default function() {
  http.get('https://app.example.com/api/health');
}
```

9. Automation & Scripting Best Practices

Q: How do you make automation scripts resilient and production-safe?

- Dry-run mode: add `--dry-run` flag that logs actions without executing them
- Idempotency: running the script twice should produce the same result (check state before acting)
- Error handling: use `try/except` (Python) or `set -e + trap` (bash). Never silently swallow errors.
- Logging: structured logs with timestamps, action taken, resource affected
- Rollback plan: for every action, define how to undo it
- Blast radius limits: never process all resources in one batch — use pagination and rate limiting
- Alerting: send SNS/Slack notification on failure with enough context to debug
- Test in dev/staging before running in production — always